

2019-2020学年秋季学期

漏洞利用与攻防实践

*Exploiting Software Vulnerability-
Techniques and Practice*

报告：李玉冰、李玥琚

实验：任泽众、范雨琳

漏洞利用与攻防实践

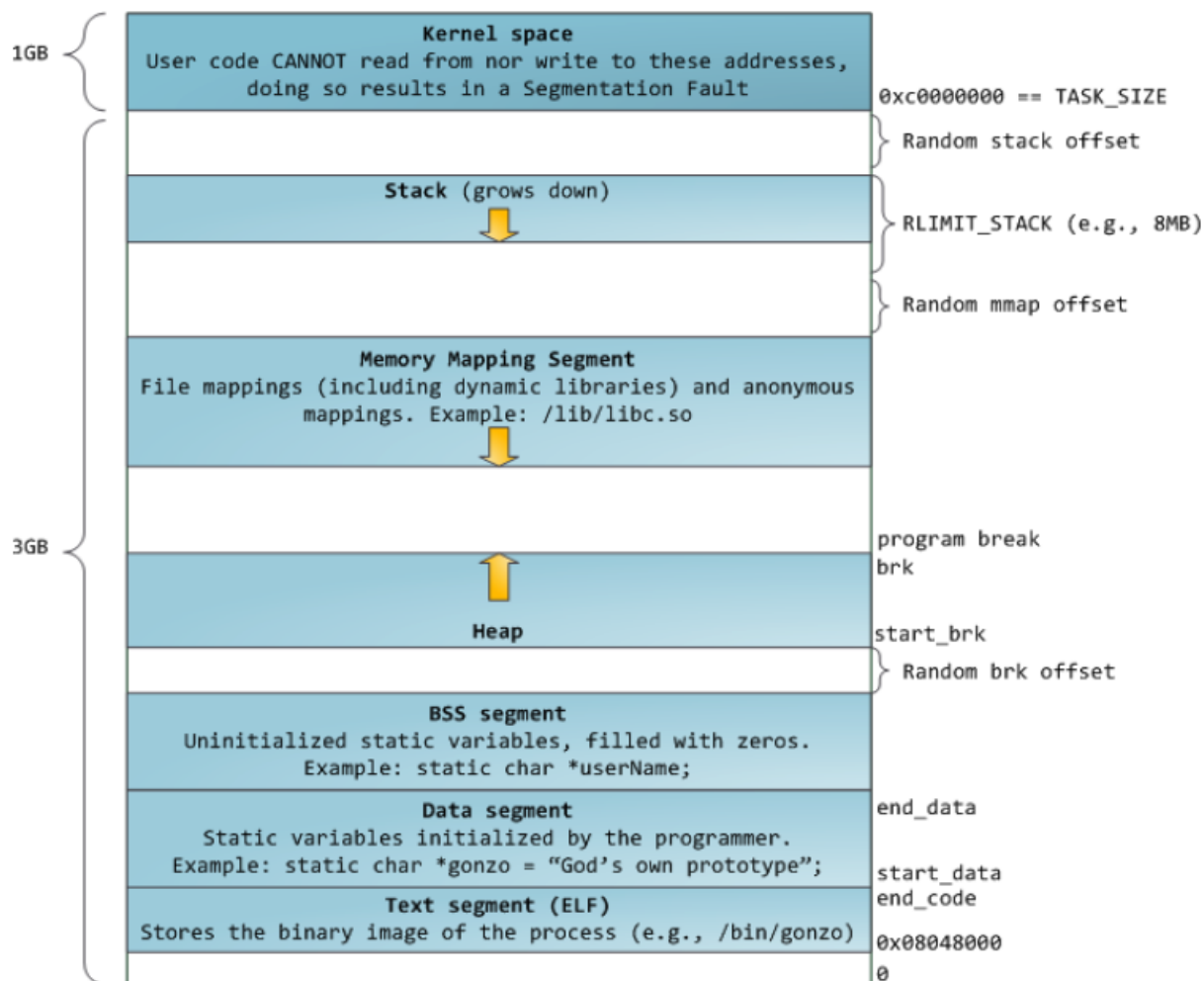
Exploiting Software Vulnerability-Techniques and Practice

[第三次课]堆利用

概要

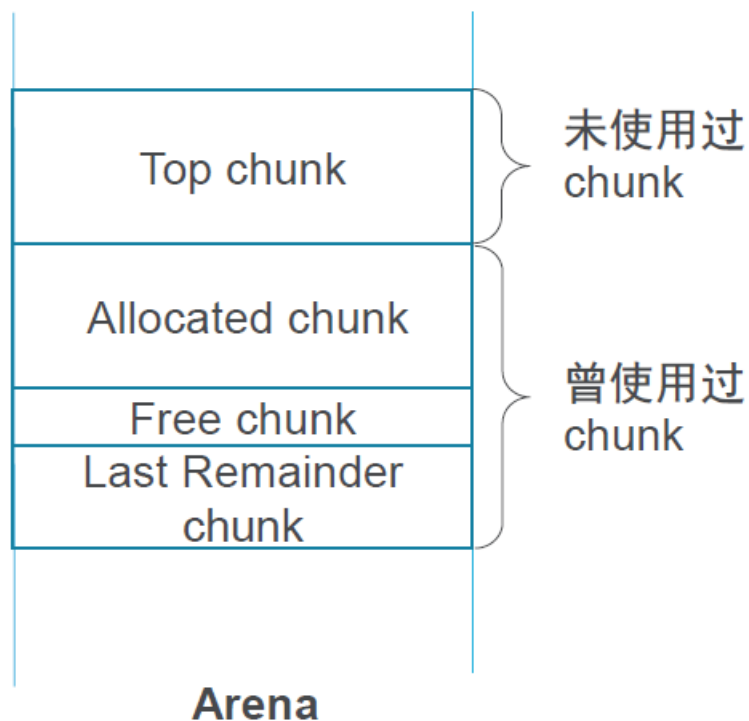
- 堆的基本知识
 - 数据结构
 - Chunk
 - 管理方式(bins)
 - 内存分配和释放函数
- 堆利用
 - 概述
 - 堆溢出
 - Double free()
 - Use After free()
 - Unlink()

堆在内存中的位置



4种chunk

- **allocated chunk**
- **free chunk**
- **top chunk**
- **last remainder chunk**



堆的数据结构

○ heap_info

- 指定堆的大小并提供指向arena数据结构的指针，主线程无此结构

○ malloc_state

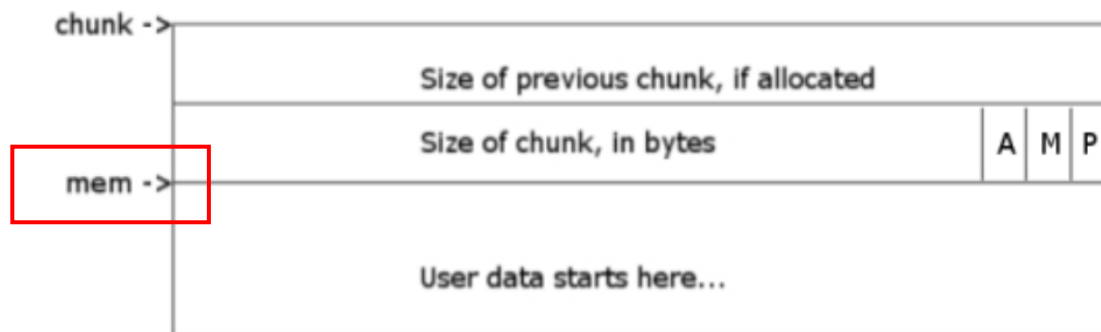
- 一个线程包含一个此结构，存储互斥锁信息、bins信息等

○ malloc_chunk

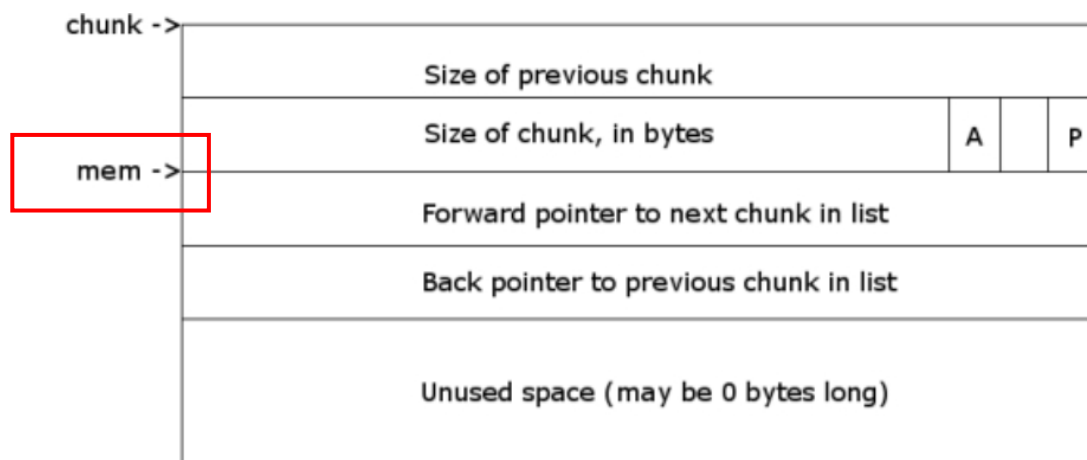
```
struct malloc_chunk {  
    INTERNAL_SIZE_T prev_size;  
    INTERNAL_SIZE_T size;  
    struct malloc_chunk * fd;  
    struct malloc_chunk * bk;  
    struct malloc_chunk * fd_nextsize;  
    struct malloc_chunk * bk_nextsize;  
}
```

堆的数据结构

allocated chunk



free chunk

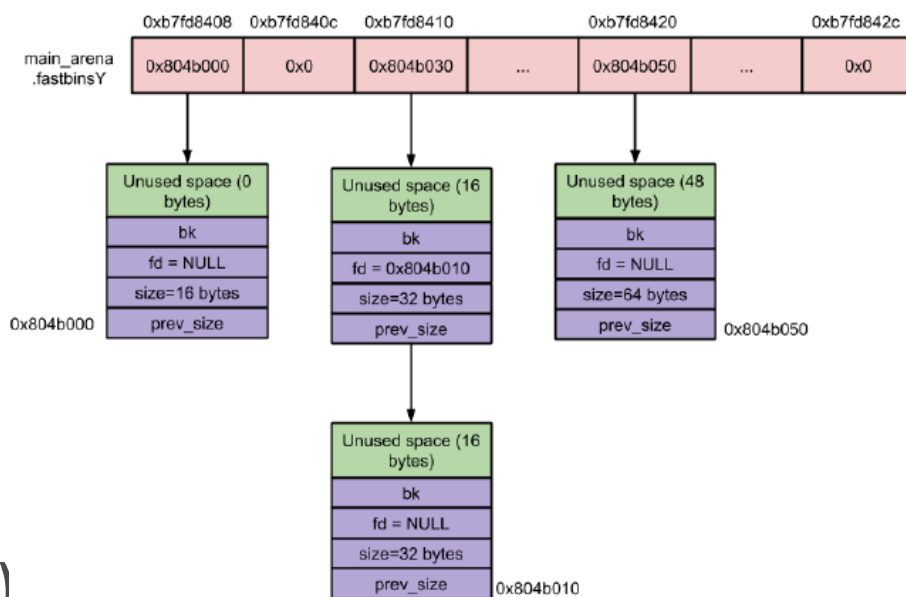


堆的管理方式(bins)

Fastbin(单向链表)

- Chunk 0~80字节

- 共10个单向链表



Normal bin(双向链表)

- small bin(chunk 小于512字节)

- large bin(chunk 大于512字节)

- unsorted bin(small bin 合并和large bin拆分后放入unsorted bin)

内存分配及释放的函数

- `void * malloc(size_t num);`
- `void * calloc(size_t num, size_t size);`
- `void * realloc(void * old, size_t new_size);`
- `void free(void * ptr);`
- **`void * alloca(size_t size);`**

分配的内存存在栈上，且会自动释放，用 `free()` 释放会导致错误。

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    int * p = (int *)alloca(sizeof(int) * 10);
    free(p);
    return 0;
}
```

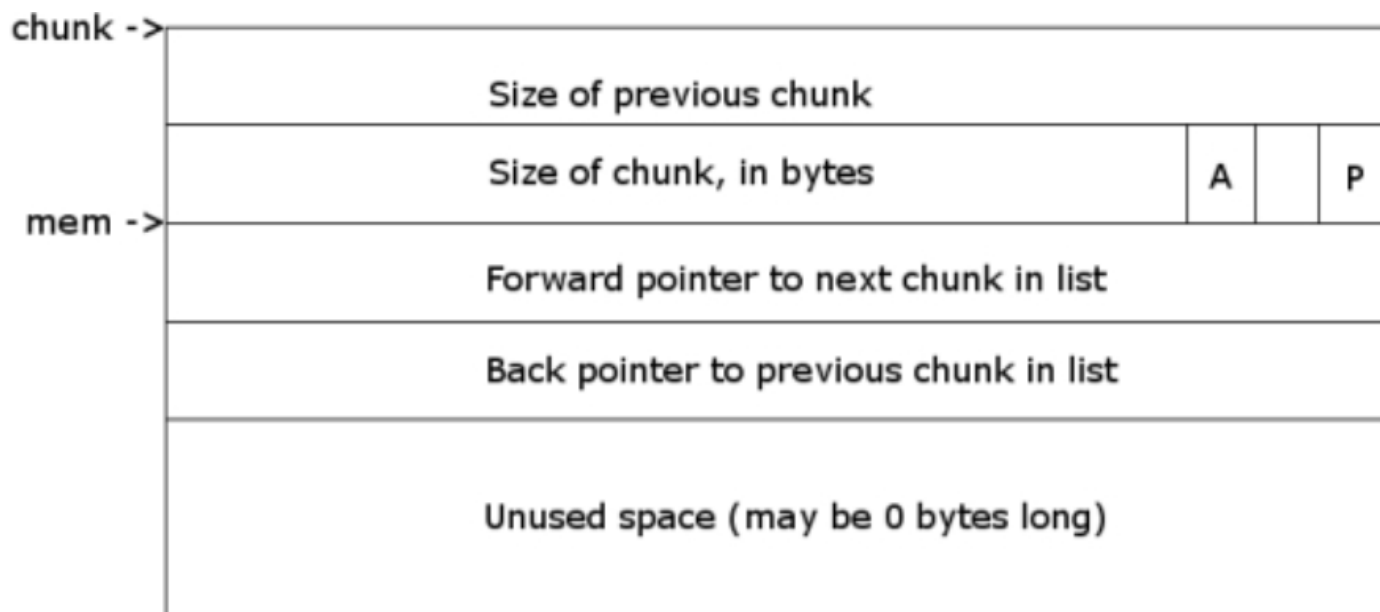
Segment default

内存分配及释放的函数

void * malloc(size_t num);

num : 请求分配的内存字节数

返回值 : 成功则返回一个指向内存的指针，否则返回NULL。



参考链接:

<https://blog.csdn.net/leisure512/article/details/4788467>

<https://blog.csdn.net/lalor/article/details/7426184>

内存分配及释放的函数

void * calloc(size_t num, size_t size);

num : 请求分配的单元个数

size : 每个单元的大小

返回值 : 如果成功则返回一个内存指针，指向一块内存已被清零的内存，失败返回NULL。

```
int *p = (int *)calloc(4, sizeof(int));
```

```
int *p = malloc(sizeof(int) * 4);  
memset(p, 0, sizeof(int) * 4);
```

内存分配及释放的函数

void * realloc(void * old, size_t new_size);

old : 原来分配的内存地址指针

new_size : 请求分配的内存的新的尺寸，可以比原来小（但是对于数据来说不安全），也可以比原来的大。

返回值 : realloc会将原来内存中的内容复制到新的内存区域，然后返回新的内存指针，失败返回NULL。

当realloc(ptr,size)的size不等于ptr的size时

如果申请size>原来size

如果chunk与top chunk相邻，直接扩展这个chunk到新size大小

如果chunk与top chunk不相邻，相当于free(ptr),malloc(new_size)

如果申请size<原来size

如果相差不足以容得下一个最小chunk(64位下32个字节，32位下16个字节)，则保持不变

如果相差可以容得下一个最小chunk，则切割原chunk为两部分，free掉后一部分

当realloc(ptr,size)的size等于0时，相当于free(ptr)

当realloc(ptr,size)的size等于ptr的size，不进行任何操作

内存分配及释放的函数

void free(void * ptr);

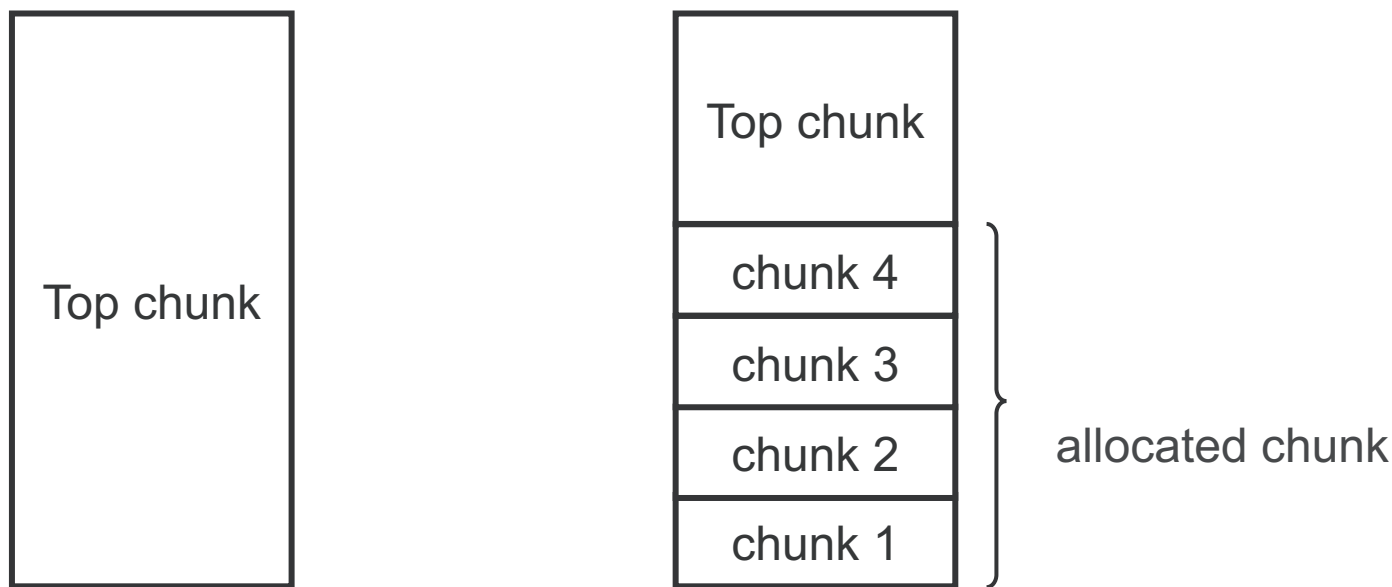
ptr : 将要释放内存空间的地址。

只能释放动态申请的内存空间，**重复释放**会出现不可预知的情况。

内存分配及释放的函数

第一次用malloc申请四块内存：

分配结果为chunk1(24B)、chunk2(24B)、chunk3(24B)、chunk4(24B)；



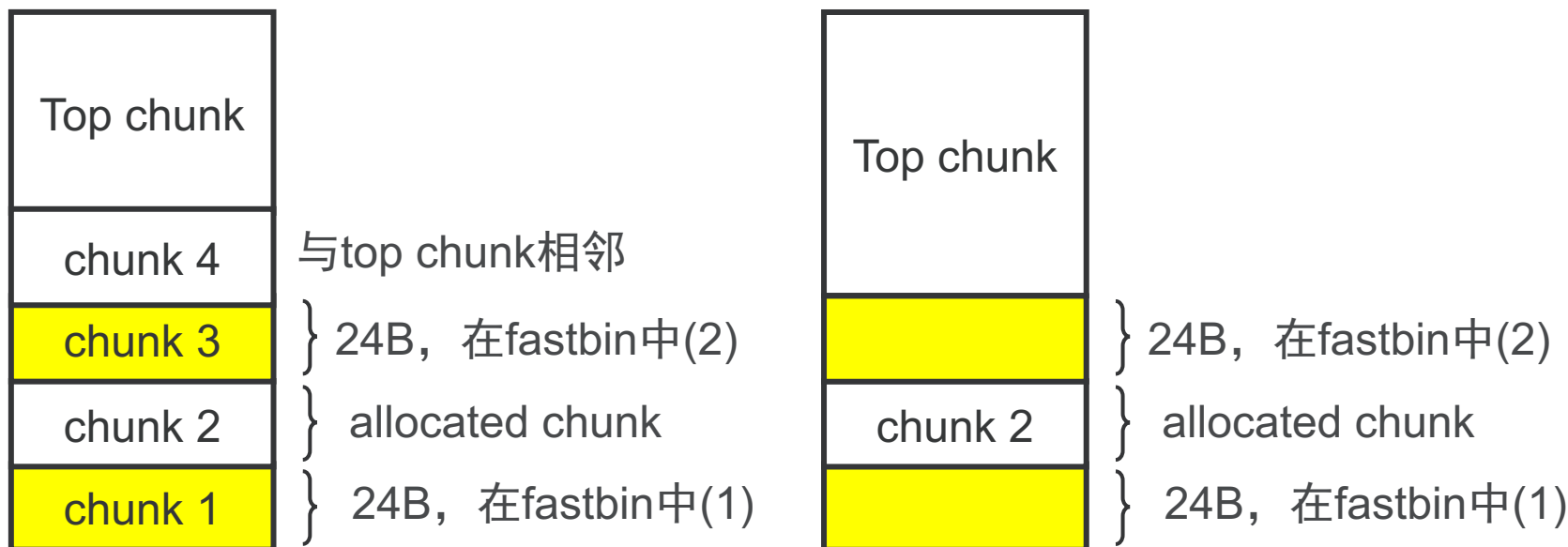
参考链接：

<https://blog.csdn.net/leisure512/article/details/4788467>

<https://blog.csdn.net/lalor/article/details/7426184>

内存分配及释放的函数

如果按顺序free(): chunk1、chunk3、chunk4;



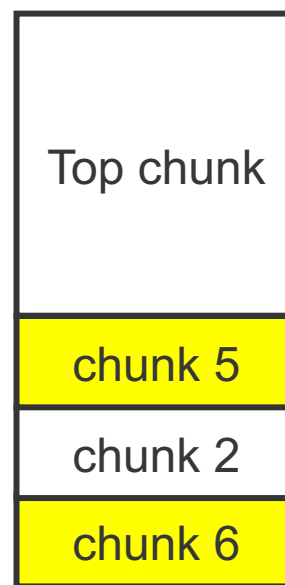
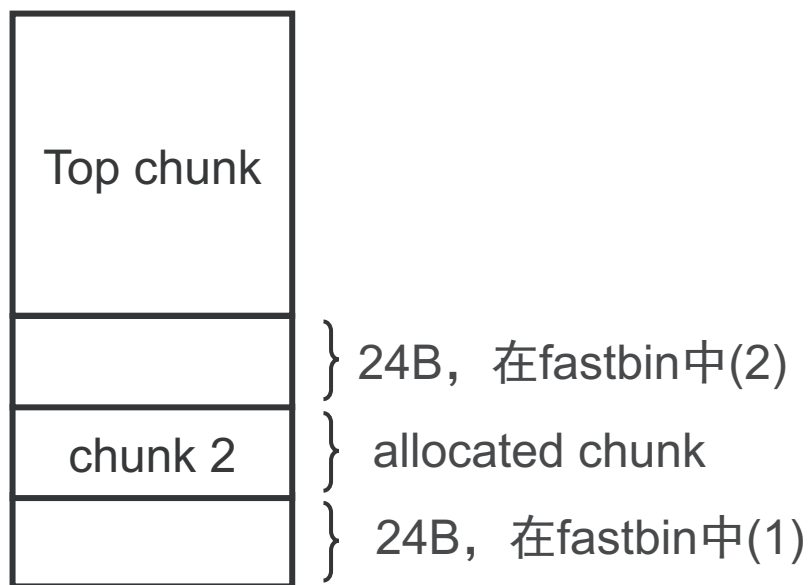
参考链接:

<https://blog.csdn.net/leisure512/article/details/4788467>

<https://blog.csdn.net/lalor/article/details/7426184>

内存分配及释放的函数

再依次申请2块内存，
分配结果为：chunk5(24B)、chunk6(24B)；
Fastbin后入先出。



参考链接：

<https://blog.csdn.net/leisure512/article/details/4788467>

<https://blog.csdn.net/lalor/article/details/7426184>

堆利用概述

Attack	Target	Technique
First Fit	这不是攻击，它只是展示了glibc分配器的本质	--
Double Free	使malloc返回已经分配的fast chunk	通过两次释放块来中断fast bin
Forging chunks	使malloc返回一个几乎任意的指针	破坏fast bin链接结构
Unlink Exploit	获得（几乎）任意写访问	释放损坏的块并利用取消链接
Shrinking Free Chunks	使malloc返回与已分配的块重叠的块	通过减小其大小来破坏自由块
House of Spirit	使malloc返回一个几乎任意的指针	强制释放精心制作的假块
House of Lore	使malloc返回一个几乎任意的指针	破坏small bin链接结构
House of Force	使malloc返回一个几乎任意的指针	溢出到顶部块的标题
House of Einherjar	使malloc返回一个几乎任意的指针	将单个字节溢出到下一个块中

堆溢出

- **堆溢出**是指程序向某个堆块中写入的字节数超过了堆块本身可使用的字节数
- 堆上并不存在返回地址等可以让攻击者直接控制执行流程的数据，因此我们一般无法直接通过堆溢出来控制 EIP。一般来说，我们利用堆溢出的策略是
 - 覆盖与其物理相邻的下一个 chunk 的元数据。
 - 利用堆中的机制（如 unlink 等）来实现任意地址写入（Write-Anything-Anywhere）或控制堆块中的内容等效果，从而来控制程序的执行流。

堆溢出

○寻找分配函数

- void * malloc(size_t num);
- void * calloc(size_t num, size_t size);
- void * realloc(void * old, size_t new_size);

○寻找危险函数

- 输入：gets，直接读取一行，忽略 '\x00'；scanf；vsanf
- 输出：sprintf
- 字符串：strcpy，字符串复制，遇到 '\x00' 停止；strcat，字符串拼接，遇到 '\x00' 停止；bcopy

快速确定程序是否可能有堆溢出，
以及有的话，堆溢出的位置在哪里

堆溢出

○ 示例

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    char *chunk;
    chunk = malloc(24);
    puts("Get input:");
    gets(chunk);
    return 0;
}
```

输入字节数远远超过可用的字节数

堆溢出

○示例结果

```
gdb-peda$ x /10gx 0x602010-0x10
0x602000: 0x0000000000000000 0x0000000000000021
0x602010: 0x0000000000000000 0x0000000000000000
0x602020: 0x0000000000000000 0x00000000000020fe1
0x602030: 0x0000000000000000 0x0000000000000000
0x602040: 0x0000000000000000 0x0000000000000000
```

```
gdb-peda$ x /10gx 0x602010-0x10
0x602000: 0x0000000000000000 0x0000000000000021
0x602010: 0x4141414141414141 0x4141414141414141
0x602020: 0x4141414141414141 0x4141414141414141
0x602030: 0x4141414141414141 0x4141414141414141
0x602040: 0x4141414141414141 0x4141414141414141
```

double free()

○申请的同一块内存释放两次

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    int * a = malloc(10);
    int * b = malloc(10);
    free(a);
    free(a);
    int * c = malloc(10);
    return 0;
}
```

有double free()检查机制

```
*** Error in `./doublefree0': double free or corruption (fasttop): 0x00000000023
96010 ***
```

double free()

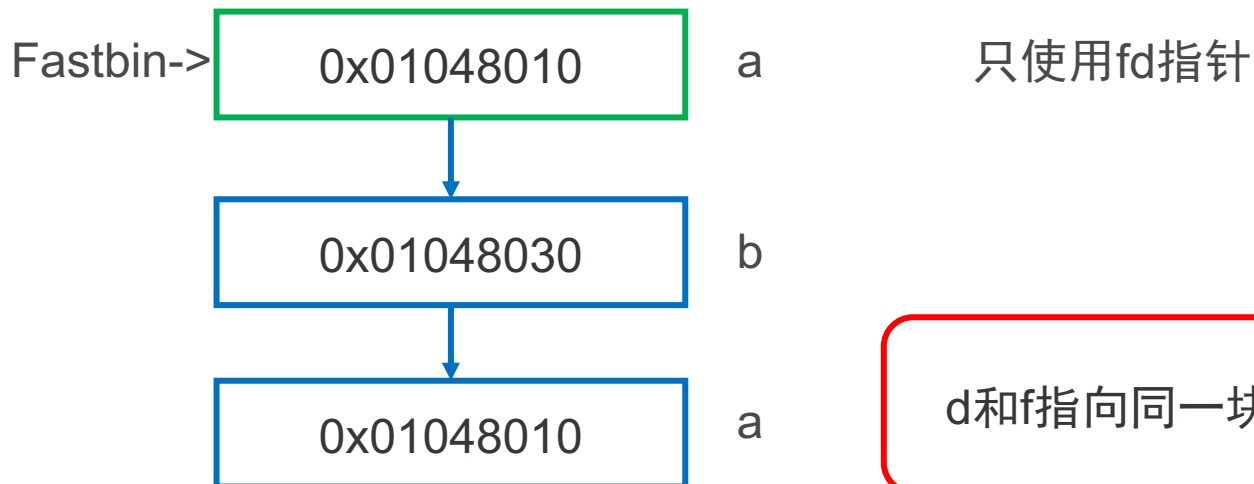
○ 示例

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    int * a = malloc(10);
    int * b = malloc(10);
    int * c = malloc(10);
    printf("Before free:\na:0x%08x\tb:0x%08x\tc:0x%08x\n", a, b, c);
    free(a);
    free(b); //To bypass "double free or corruption" check
    free(a); //Double free
    int * d = malloc(10);
    int * e = malloc(10);
    int * f = malloc(10);
    printf("After free:\nd:0x%08x\te:0x%08x\tf:0x%08x\n", d, e, f);
    return 0;
}
```

正在释放的指针不能是
fastbin链表的最后一项

double free()

○示例



d和f指向同一块内存

Before free:

a:0x01048010

b:0x01048030

c:0x01048050

After free:

d:0x01048010

e:0x01048030

f:0x01048010

double free()

○ double free()能用来做什么？



稍后实验演示

User after free

- Use After Free 就是其字面所表达的意思，当一个内存块被释放之后再次被使用。但是其实这里有以下几种情况
 - 内存块被释放后，其对应的指针被设置为 NULL，然后再次使用，自然程序会崩溃。
 - 内存块被释放后，其对应的指针没有被设置为 NULL，然后在它下一次被使用之前，没有代码对这块内存块进行修改，那么程序很有可能可以正常运转。
 - 内存块被释放后，其对应的指针没有被设置为 NULL，但是在它下一次使用之前，有代码对这块内存进行了修改，那么当程序再次使用这块内存时，就很有可能会出现奇怪的问题。
- 而我们一般所指的 Use After Free 漏洞主要是后两种。此外，我们一般称被释放后没有被设置为 NULL 的内存指针为 dangling pointer(野指针)。

User after free

○ 示例

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
int main()
{
    char * p1, * p2;
    p1 = (char *)malloc(sizeof(char) * 10);
    memcpy(p1, "hello", 10);
    printf("p1 addr: 0x%08x, %s\n", p1, p1);
    free(p1); //not Set NULL
    p2 = (char *)malloc(sizeof(char) * 10);
    memcpy(p2, "world", 10);
    printf("p2 addr: 0x%08x, %s\n", p2, p1);
    return 0;
}
```

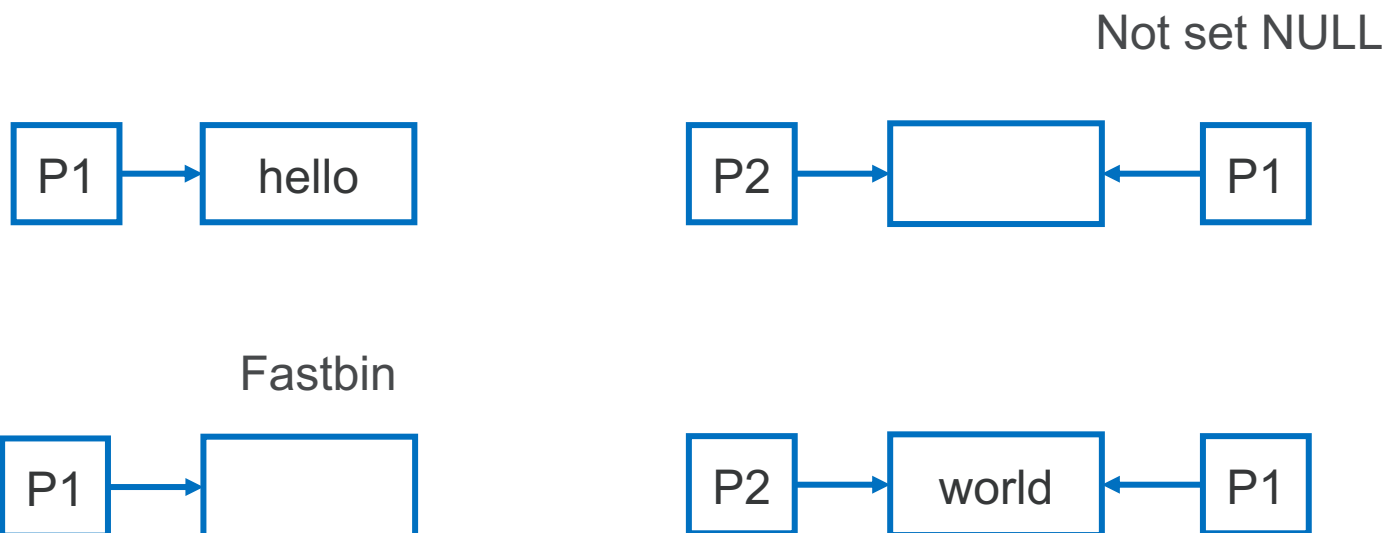
参考链接:

<https://blog.csdn.net/guilanl/article/details/64906474>

https://blog.csdn.net/qq_27312943/article/details/53152244

User after free

○ 示例



```
p1 addr: 0x01546010, hello  
p2 addr: 0x01546010, world
```

可以通过p2, 修改p1。

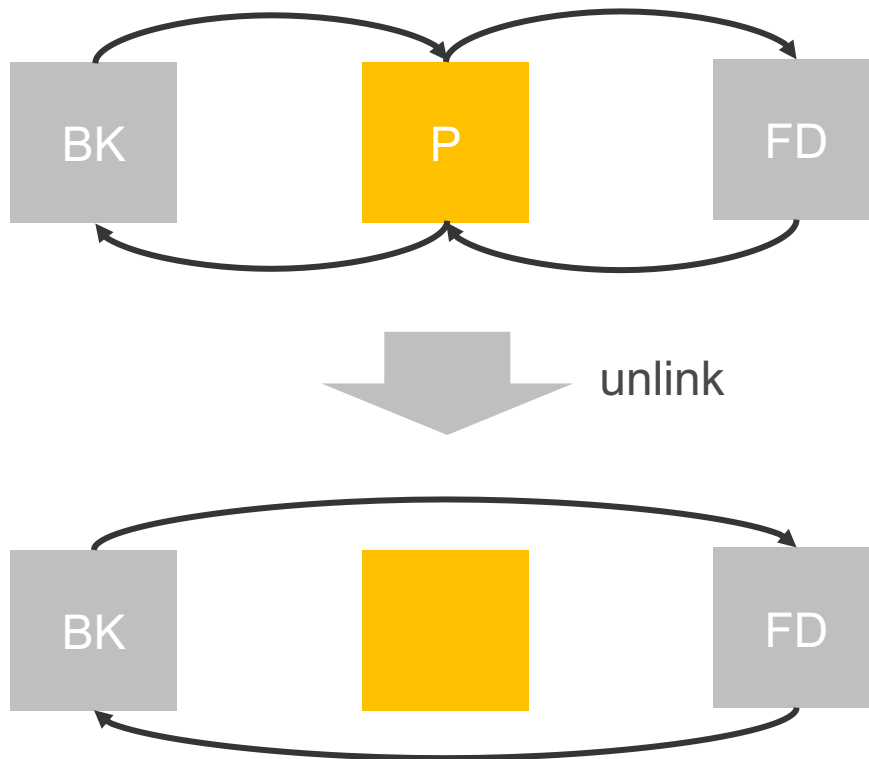
参考链接:

<https://blog.csdn.net/guilanl/article/details/64906474>

https://blog.csdn.net/qg_27312943/article/details/53152244

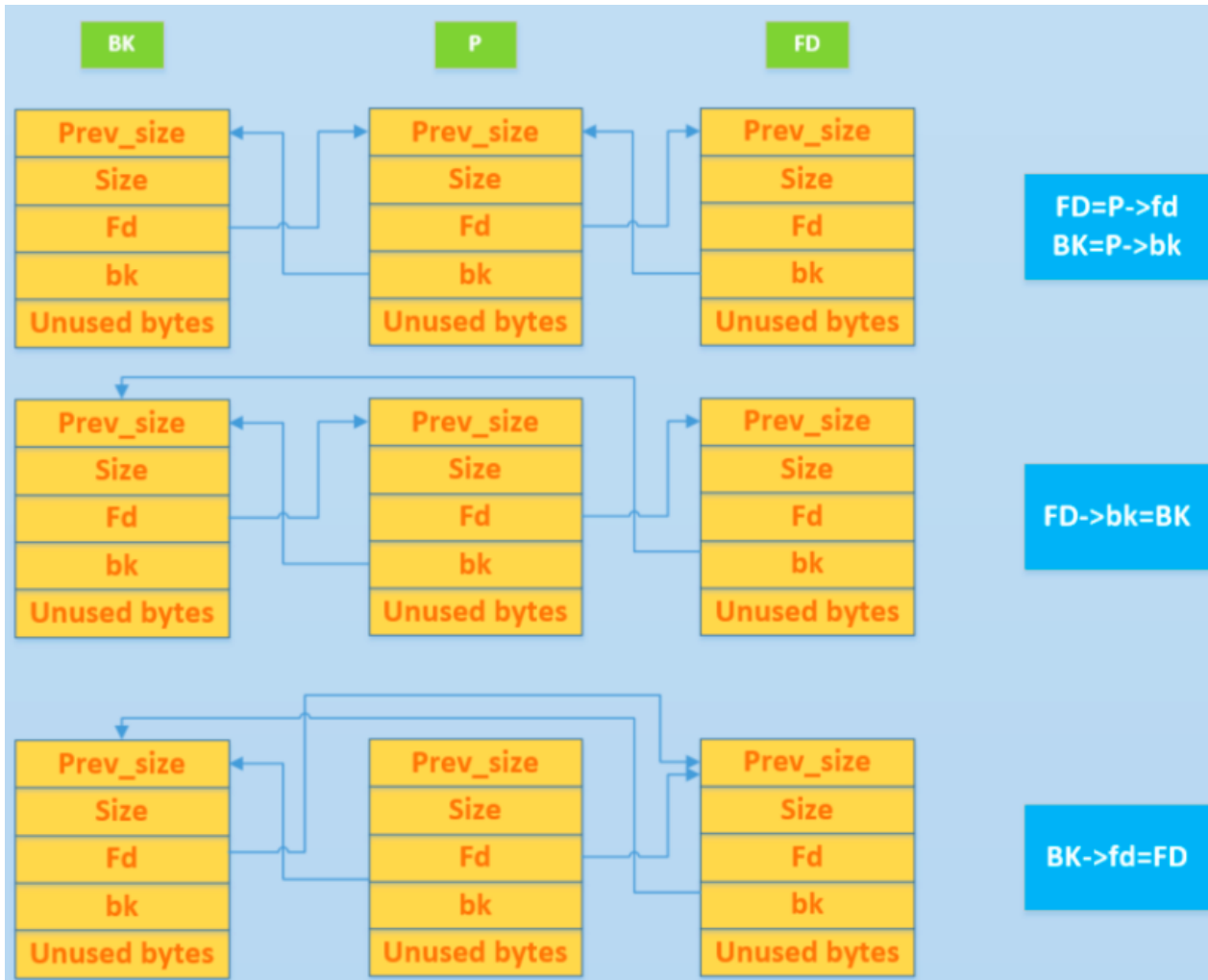
Unlink

Unlink是一个宏；
当某个chunk从双链表中脱节点时，会调用unlink。

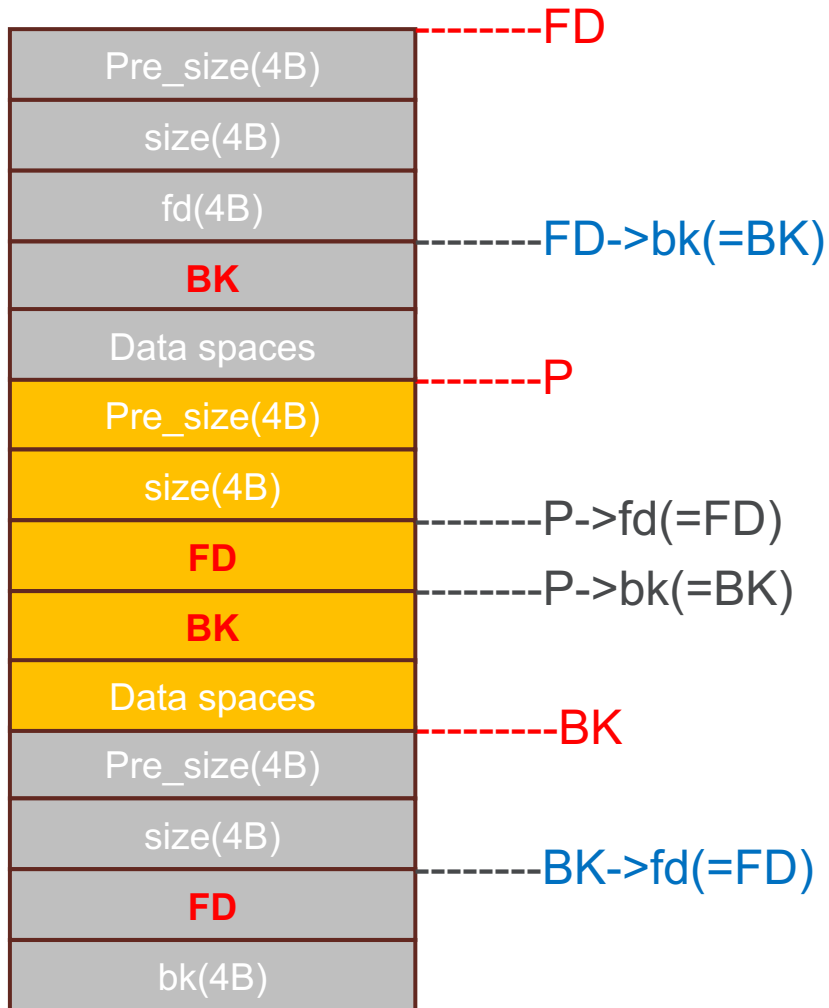


unlink漏洞的结果是
在任意的可写地址写
入任意你想写的内容

Unlink



Unlink



$BK = P \rightarrow bk$
 $FD = P \rightarrow fd$
 $FD \rightarrow bk = BK$
 $BK \rightarrow fd = FD$



$FD = *(P + 8)$
 $BK = *(P + 12)$

$*(FD + 12) = BK$
 $*(BK + 8) = FD$

有防护机制的Unlink

```
#define unlink(P, BK, FD){  
    FD=P->fd;  
    BK=P->bk;  
    if(__builtin_expect(FD->bk!=P || BK->fd!=P,0))  
        malloc_printerr(check_action, "corrupted double-linked list", P);  
    else{  
        FD->bk=BK  
        BK->fd=FD;  
    }  
}
```

检测该节点的fd,bk是否构成双向链表

即满足：

(1) $P \rightarrow fd \rightarrow bk = P$ (2) $P \rightarrow bk \rightarrow fd = P$

问题

Q&A